React Fundamentals

Daily Revision Notes • Before continuing Hitesh Choudhary's Video 11 (Currency Converter)

How to use this PDF: Spend 5–10 minutes revising the highlighted rules, then explain each example in your own words without looking.

1. Components and JSX

A **component** is a JavaScript function that returns UI. Defining a function does not display it by itself; it must be used in the rendered component tree.

Example:

```
function Welcome() {
return <h1>Hello!</h1>;
}

function App() {
return <>
<Welcome />
</>;
}
```

Fragment: `<>...</>` groups elements without adding an extra HTML wrapper.

2. Props: passing data from parent to child

The parent passes values to a component using attributes in JSX. The child receives them through **props**.

```
function Welcome(props) {
return <h1>Hello {props.name}</h1>;
}

<Welcome name="Mustafa" />
```

Multiple instances can receive different props: `<Welcome name="Ali" />` and `<Welcome name="Sara" />` render different names.

Destructuring: `function Welcome({ name, age })` extracts properties from the props object. It is ordinary JavaScript syntax.

JSX values: `name="Mustafa"` passes a string. `age={21}` passes a number.

3. Events: when functions run

Correct: `onClick={sayHello}` gives React the function to run when clicked.

Usually wrong for click handling: `onClick={sayHello()}` calls the function immediately during render instead of waiting for the click.

For a text input, `onChange` runs as the input value changes.

```
function handleChange(event) {
console.log(event.target.value);
}
```

Important: `event.target.value` is the entire current input text, not only the newest character. Typing C, then A, then T logs C, CA, CAT. To get the length, use `event.target.value.length`.

4. useState: state, updates, and rendering

`useState` lets a component remember a value between renders. It returns the current state and a setter function.

```
const [name, setName] = useState("");
```

```
return <>
<input onChange={(event) => setName(event.target.value)} />
<p>{name}</p>
</>;
```

Trace it: typing “Mustafa” calls `setName("Mustafa")`; React schedules a re-render; `<p>{name}</p>` then displays “Mustafa”. Clearing the input makes the state an empty string again.

State update causes a re-render. A normal variable changing does not, by itself, tell React to re-render the UI. If an input handler only logs the value and never updates state, the console changes but the state-driven paragraph remains unchanged.

Do not reassign the state variable: with `const [count, setCount] = useState(0)`, use `setCount(...)` to request an update. `count = count + 1` is not the correct approach (and reassignment to this const binding throws an error).

5. State updates: direct values vs updater functions

Example A — two direct updates:

```
const [count, setCount] = useState(0);
function increase() {
setCount(count + 1);
setCount(count + 1);
}
```

Starting at 0, the result is **1**, not 2. Both expressions use `count` from the same current render, so both queue the value 1.

Example B — updater functions:

```
function increase() {
setCount(prev => prev + 1);
setCount(prev => prev + 1);
}
```

Starting at 0, the result is **2**. React applies each updater to the latest queued state: 0 → 1 → 2.

MAIN RULE: `setCount(count + 1)` calculates from the **current render’s count**. `setCount(prev => prev + 1)` calculates from the **latest queued state**. Use the updater form when the next value depends on the previous value.

6. Mixed-update examples we practised

Starting count = 5:

Updates in order	Final count	Why
prev + 1; prev + 1; count + 3	**8**	The last direct update uses original count 5, so it queues 5 + 3 = 8.
prev + 1; count + 3; prev + 1	**9**	6 is queued, direct update replaces it with 8, then updater makes 9.

Do not just memorize the answers. For each update, ask: Is this a direct value based on the current render, or an updater function based on the latest queued state?

7. Quick daily self-test

1. What is the difference between defining a component and rendering it?

2. What are props, and how does a child read the name prop?

3. What does destructuring do in function Welcome({ name })?

4. Why is onClick={sayHello} different from onClick={sayHello()}?

5. What does event.target.value contain while typing CAT?

6. Why does setName(event.target.value) update the UI, but a normal variable assignment usually does not?

7. Starting at 0, what do two setCount(count + 1) calls produce, and why?

8. Starting at 0, what do two setCount(prev => prev + 1) calls produce, and why?

9. When should you prefer the updater-function form?

Answers to check after trying: components must be used in the rendered tree; props carry parent-to-child data; destructuring extracts a property; the first event form passes a function and the second calls it immediately; event.target.value is the full current text; state setters trigger React updates; two direct calls from count 0 yield 1; two updater calls yield 2; prefer updater functions when the next value depends on previous state.

Next step

Once these notes feel familiar, continue with a small lesson on updating objects in state, then return to Hitesh Choudhary's Video 11 (Currency Converter). The goal is to understand where each value comes from and why each update happens—not to memorize the project code.